

SD TCL - The Command Stack

Recalling and re-running commands, the @variables, and the session verbs.

This page continues SD TCL - The Command Processor.

The command stack

Every line you type is kept. The stack is edited with commands beginning with a dot, and `n` below is a stack position — 1 is the most recent.

<code>.l {n}</code>	list the last <i>n</i> stack lines. Defaults to 20, or a screenful
<code>.l name</code>	show a VOC record, one numbered line per field
<code>.r {n}</code>	recall line <i>n</i> to the top of the stack
<code>.r name</code>	load a sentence or paragraph from the VOC into the stack
<code>.s name s e</code>	save stack lines <i>s</i> to <i>e</i> into the VOC as <i>name</i>
<code>.a {n} text</code>	append <i>text</i> to line <i>n</i>
<code>.i {n} text</code>	insert <i>text</i> as line <i>n</i>
<code>.c {n} / old / new / {g}</code>	change <i>old</i> to <i>new</i> in line <i>n</i> ; <i>g</i> for every occurrence
<code>.d {n}</code>	delete line <i>n</i>
<code>.d name</code>	delete a sentence or paragraph from the VOC, after asking
<code>.u {n}</code>	convert line <i>n</i> to upper case
<code>.x {n}</code>	execute line <i>n</i>
<code>.x file id</code>	execute a sentence or paragraph held in any file, not just the VOC
<code>.?</code>	print this list

`n` defaults to 1 everywhere, and the spaces shown are required. `.d2` deletes the second stack line and `.d 2` looks for a VOC record called `2`.

`.s` and `.r` together are the whole workflow: type the lines, then save them under a name, then recall them later to edit. `.s` writes a sentence when the range is one line and a paragraph when it is more, so the record type follows from what you saved rather than from anything you say.

`.x` file id is the one that reaches outside the VOC — the file name and the record id are each folded as typed, then lower, then upper.

A dot command that is not one of these is not an error. It is left alone and run as an ordinary command, so a verb whose name begins with a dot still works.

`.d` does not find a lower-case VOC record typed in upper case, and `.l` and `.r` do. `.l` and `.r` try the name as typed, then lower case, then upper case. `.d` tries only as typed and then upper case. A paragraph saved as `daily` can be listed and recalled by typing `DAILY` and cannot be deleted by typing `DAILY`. Type the name in the case you saved it in and all three work.

EDITING THE LINE YOU ARE ON

There are two help texts and they are different. `?` on its own prints the key map below; `.?` prints the dot-command list above. Neither is the `help` verb, which does not exist — see *What is not here*.

The key map is the same whether you reached the line by typing it or by recalling it:

KEY		ALSO
<code>Ctrl-A</code>	start of line	Home
<code>Ctrl-E</code>	end of line	End
<code>Ctrl-B</code> / <code>Ctrl-F</code>	back / forward one character	Cursor left / right
<code>Ctrl-P</code> / <code>Ctrl-N</code>	previous / next command	Cursor up / down
<code>Ctrl-Z</code>	previous command	Cursor up
<code>Ctrl-R</code>	reverse search	
<code>Ctrl-D</code>	delete character	Delete
<code>Ctrl-H</code>	backspace	Backspace
<code>Ctrl-K</code>	kill line	
<code>Ctrl-O</code>	toggle overlay	Insert
<code>Ctrl-T</code>	transpose characters	

KEY

ALSO

<code>Ctrl-U</code>	upcase all
<code>Ctrl-G</code>	exit the stack

A line ending in `?` is put on the stack without being run. That is the way to park a half-written command while you go and look something up.

SAVING THE STACK ACROSS SESSIONS

<code>save.stack {name}</code>	write the stack to a named record
<code>get.stack {name}</code>	read it back
<code>clear.stack</code>	empty it

Both prompt for the name if you leave it off.

@-variables

`set` assigns an @-variable, which is then substituted into later command lines wherever its name appears.

```
set name value
set name eval expression
```

Without `eval` the value is taken literally. With `eval` the text is evaluated first, and the evaluation handles the four arithmetic operators on numeric operands. `list.vars` shows what is currently set.

The variables the system sets are readable the same way — `@system.return.code` is the one worth knowing, because most verbs set it and `report.src` exists to show it.

Session verbs

<code>who</code>	your user number and the account you are in
<code>who.am.i</code>	the same information at more length

<code>logto</code> <i>account</i>	change account without logging out
<code>off</code> · <code>quit</code>	end the session
<code>option</code> { <i>name</i> } { <code>on off display</code> }	set or show a session option
<code>option all off</code>	turn every session option off at once
<code>alias</code> <i>command target</i>	make <i>command</i> run as <i>target</i>
<code>alias</code> <i>command</i>	remove that alias
<code>alias</code>	list the aliases
<code>clear.abort</code>	clear a pending abort condition
<code>set.exit.status</code> <i>n</i>	set the numeric status SD exits with
<code>report.src</code> { <code>on off</code> }	show <code>@system.return.code</code> after each command

`who` answers with your user number and the account you are in:

```
39 DON
```

After a `logto` it grows a third part, and that is the useful one:

```
29 SDSYS from DON
```

`from DON` is the account you logged in as, not the one you are in and not your Windows account. So the short form means *I am still where I started* and the long form means *I have moved* — which makes `who` the quick way to find out whether a `logto` actually took effect.

`logto sdsys` **requires an elevated Windows session in this port.** Entering `SDSYS` is what confers administrator rights, so it is gated on the operating system rather than on an SD password, and the elevation obtained by one `logto` is deliberately not carried into the next one. `who` reports the account you are in, which is the quick way to confirm a `logto` actually happened.

What is not here

There is no online help and `help` is not a verb. The command processor still has internal verb 14 reserved for it and the routine is an empty stub — its body is commented out and it returns immediately. **No VOC record points at it**, in any account type, so the name is not even recognised. The F1 key at the command prompt reaches the same empty

routine and therefore does nothing. **This documentation is the help system.**

`umask` **is implemented and unreachable.** Internal verb 35 is a working routine that reports or sets the file-creation mask, and **no VOC record points at it either.** It cannot be typed. `umask()` from SD BASIC still works — see SD Basic - System and Environment.

PROC is removed. So are `sed` and `update.record`. A `PQ`-type VOC record is **refused by name** rather than being reported as a bad dispatch code, because the record itself is valid PROC and it is the interpreter that is gone. Nothing that ships is type `PQ`, so this can only be met in a VOC record somebody wrote.

`menu` **records are dispatched but no menu ships.** Type `M` is handled and there is no shipped example to look at.

Who has these verbs

Everything on this page is in a **standard** account except the last two, which are administrator-only:

standard

`abort` `alias` `clear.abort` `clear.stack` `display` `get.stack` `go` `if` `list.vars` `logto`
`off` `option` `pause` `quit` `report.src` `save.stack` `set` `set.exit.status` `stop` `who`
`who.am.i`

Everything on this page is in a standard account. An account that does not have a verb does not have the VOC record for it — the name is simply not recognised rather than refused.

The two `OS` verbs are the exception and are not documented here. `sh` and `!` reach the Windows shell, are administrator-tier, and are gated a second time by a list of who may use them. They are in the **administrator documentation**, under *Operating System Access*, which is a separate set.

See also

SD TCL - Files and Records · SD Basic - Program Control · SD Basic - System and Environment.